

UniApp iOS 原生插件开发全流程详解

UniApp iOS 原生插件开发全流程详解

UniApp 原生插件开发（iOS 端）核心是通过 Objective-C/Swift 编写原生代码，封装成 UniApp 可调用的插件，弥补 JS 层无法直接操作 iOS 系统底层、硬件或第三方原生 SDK 的短板。以下从**开发环境、插件结构、核心开发步骤、调试发布**等维度，详细拆解全流程。

一、开发前准备

1. 环境要求

工具/环境	版本要求	说明
macOS 系统	macOS 10.15+	必须用 macOS 开发 iOS 原生插件
Xcode	12.0+（建议 14.x/15.x）	iOS 原生开发核心工具
HBuilderX	3.0+	编写 UniApp 代码、打包插件
iOS 真机/模拟器	iOS 11.0+	调试插件（真机调试需开发者账号）
Pod（可选）	1.10+	集成第三方 iOS SDK 时使用

2. 核心概念前置

- **UniApp 插件类型：**
 - 「模块插件」：无 UI，提供纯功能调用（如支付、定位、蓝牙）；
 - 「组件插件」：带 UI 渲染（如自定义导航栏、原生图表）；
 - 本文以「模块插件」为核心讲解（组件插件仅多 UI 渲染逻辑）。
- **通信逻辑：**JS 层 → UniApp 桥接层（UniMP） → iOS 原生插件 → 回调 JS 层。
- **插件目录规范：**UniApp 插件需遵循固定目录结构，否则无法被识别。

二、创建 iOS 原生插件工程

1. 初始化插件骨架（两种方式）

方式 1: HBuilderX 生成插件模板（推荐）

1. 打开 HBuilderX → 菜单栏「文件」→「新建」→「原生插件工程」；
2. 选择「iOS 平台」→ 填写插件名称（如 `UniTestPlugin`）、插件标识（唯一，如 `com.xxx.test`）、作者等；
3. 选择保存路径，HBuilderX 会自动生成包含 iOS 工程、插件配置文件的完整骨架。

方式 2: 手动创建 Xcode 工程（进阶）

若需深度定制，可手动创建 Xcode 静态库工程：

1. 打开 Xcode → 「Create a new Xcode project」→ 选择「Framework」→ 语言选「Objective-C」（Swift 需额外配置桥接）；
2. 工程命名为插件名（如 `UniTestPlugin`），设置「Bundle Identifier」与插件标识一致；
3. 手动添加 UniApp 桥接头文件（后续步骤）。

2. 插件目录结构解析（HBuilderX 生成模板）

Code block

```
1  UniTestPlugin/
2  |—— ios/                                // iOS 原生代码目录
3  |   |—— UniTestPlugin/                  // Xcode 工程目录
4  |   |   |—— UniTestPlugin/              // 原生代码目录
5  |   |   |   |—— Classes/                 // 核心代码 (.h/.m/.swift)
6  |   |   |   |—— Resources/               // 资源文件 (图片、xib、plist 等)
7  |   |   |   |—— UniTestPlugin.xcodeproj // Xcode 工程文件
8  |   |   |   |—— Podfile (可选) | // 第三方 SDK 依赖
9  |   |   |—— plugin.json                 // 插件配置 (核心! UniApp 识别插件的关键)
10 |—— package.json                       // 插件 npm 包配置 (发布用)
```

三、核心开发步骤

步骤 1: 配置 plugin.json（插件入口声明）

`plugin.json` 是 UniApp 与原生插件的「桥梁配置」，需声明插件类型、入口类、方法名、参数等，示例如下：

Code block

```
1  {
2    "name": "UniTestPlugin",              // 插件名称 (与工程名一致)
3    "id": "com.xxx.test",                 // 插件唯一标识 (UniApp 调用时需用)
4    "version": "1.0.0",
5    "description": "iOS 测试插件",
6  }
```

```

6      "ios": {
7          "plugins": [
8              {
9                  "type": "module",          // 插件类型: module (模块) /component (组件)
10                 "name": "UniTestPlugin",  // 模块名 (JS 调用时的标识)
11                 "class": "UniTestPlugin"  // 原生入口类名 (必须与 Xcode 中类名一致)
12             }
13         ],
14         "frameworks": [                  // 依赖的 iOS 系统框架 (按需添加)
15             "UIKit",
16             "CoreLocation",
17             "AVFoundation"
18         ],
19         "pods": [                        // 依赖的 CocoaPods 库 (可选, 如 AFNetworking)
20             "AFNetworking ~> 4.0"
21         ]
22     }
23 }

```

步骤 2: 编写 iOS 原生插件代码

1. 引入 UniApp 桥接头文件

Xcode 工程中需先引入 UniApp 提供的桥接库, 才能实现与 JS 通信:

- 若用 HBuilderX 模板, 已自动引入 `UniMPiOSSDK.h`;
- 手动工程需下载 UniApp 原生 SDK ([官网下载](#)), 将 `UniMPiOSSDK.framework` 导入工程, 并在头文件中引入:

Code block

```
1  #import <UniMPiOSSDK/UniMPiOSSDK.h>
```

2. 编写入口类 (核心)

创建 `UniTestPlugin.h` 和 `UniTestPlugin.m`, 需继承 `UniModule` (UniApp 规定的模块基类), 并遵循 `UniModuleProtocol` 协议。

示例 1: 基础方法 (无回调)

Code block

```

1  // UniTestPlugin.h
2  #import <UniMPiOSSDK/UniModule.h>
3
4  @interface UniTestPlugin : UniModule

```

```

5 // 声明 JS 可调用的方法
6 - (void)sayHello:(NSDictionary *)params;
7 @end
8
9 // UniTestPlugin.m
10 #import "UniTestPlugin.h"
11
12 @implementation UniTestPlugin
13
14 // 实现方法: JS 调用时触发
15 - (void)sayHello:(NSDictionary *)params {
16     // 1. 获取 JS 传入的参数
17     NSString *name = params[@"name"] ?: @"默认名称";
18
19     // 2. 执行原生逻辑 (示例: 打印日志)
20     NSLog(@"Hello %@, 这是 iOS 原生插件输出", name);
21
22     // 3. 无回调: 直接执行逻辑即可
23 }
24
25 @end

```

示例 2: 带回调的方法 (异步返回结果给 JS)

UniApp 原生插件通过 `UniModuleCallback` 实现异步回调, 示例:

Code block

```

1 // UniTestPlugin.h
2 - (void)getDeviceInfo:(NSDictionary *)params callback:
   (UniModuleCallback)callback;
3
4 // UniTestPlugin.m
5 - (void)getDeviceInfo:(NSDictionary *)params callback:
   (UniModuleCallback)callback {
6     // 1. 获取 iOS 设备信息 (原生逻辑)
7     UIDevice *device = [UIDevice currentDevice];
8     NSDictionary *deviceInfo = @{
9         @"model": device.model,           // 设备型号 (如 iPhone 14)
10        @"systemVersion": device.systemVersion, // 系统版本 (如 17.0)
11        @"uuid": [[NSUUID UUID] UUIDString] // 设备唯一标识
12    };
13
14    // 2. 回调给 JS (成功/失败)
15    if (callback) {
16        // 成功回调: 第一个参数为 error (nil), 第二个为结果
17        callback(nil, deviceInfo);
18    }
19 }

```

```

18
19         // 失败回调示例:
20         // NSError *error = [NSError errorWithDomain:@"com.xxx.test" code:-1
userInfo:@{NSLocalizedDescriptionKey:@"获取设备信息失败"}];
21         // callback(error, nil);
22     }
23 }

```

3. Swift 开发插件（兼容处理）

若用 Swift 编写，需额外配置：

1. Xcode 工程中创建「桥接头文件」（`UniTestPlugin-Bridging-Header.h`），引入 UniApp 头文件：

Code block

```

1  #import <UniMPiOSSDK/UniModule.h>
2  #import <UniMPiOSSDK/UniModuleProtocol.h>

```

2. Swift 类需继承 `NSObject` 并遵循 `UniModuleProtocol`：

Code block

```

1  import UIKit
2
3  @objc(UniTestPlugin) // 必须加 @objc, 否则 JS 无法识别
4  class UniTestPlugin: NSObject, UniModuleProtocol {
5      @objc func sayHello(_ params: [String: Any]?) {
6          let name = params?["name"] as? String ?? "默认名称"
7          print("Hello \(name), Swift 原生插件输出")
8      }
9
10     @objc func getDeviceInfo(_ params: [String: Any]?, callback:
UniModuleCallback?) {
11         let device = UIDevice.current
12         let deviceInfo: [String: Any] = [
13             "model": device.model,
14             "systemVersion": device.systemVersion
15         ]
16         callback?(nil, deviceInfo)
17     }
18 }

```

3. `plugin.json` 中 `class` 字段需填写 Swift 类的 `@objc` 别名（如 `UniTestPlugin`）。

步骤 3: 集成第三方 SDK（可选，如 AFNetworking）

1. 在 Xcode 工程根目录创建 `Podfile`：

Code block

```
1 platform :ios, '11.0'
2 target 'UniTestPlugin' do
3     use_frameworks!
4     pod 'AFNetworking', '~> 4.0'
5 end
```

2. 终端执行 `pod install`，安装依赖后打开 `.xcworkspace` 工程（而非 `.xcodeproj`）；
3. 在原生代码中引入并使用第三方 SDK：

Code block

```
1 #import <AFNetworking/AFNetworking.h>
2
3 - (void)requestData:(NSDictionary *)params callback:
    (UniModuleCallback)callback {
4     NSString *url = params[@"url"];
5     AFHTTPSessionManager *manager = [AFHTTPSessionManager manager];
6     [manager GET:url parameters:nil progress:nil
    success:^(NSURLSessionDataTask * _Nonnull task, id _Nullable
    responseObject) {
7         callback(nil, responseObject);
8     } failure:^(NSURLSessionDataTask * _Nullable task, NSError * _Nonnull
    error) {
9         callback(error, nil);
10    }];
11 }
```

四、UniApp 端调用插件

1. 本地引入插件

1. 在 UniApp 项目根目录创建 `nativeplugins` 文件夹；
2. 将完整的插件工程（如 `UniTestPlugin`）复制到 `nativeplugins` 下；
3. 在 `pages.json` 中配置插件（仅组件插件需配置，模块插件无需）：

Code block

```
1 {
```

```
2     "plugins": {
3       "UniTestPlugin": {
4         "version": "1.0.0",
5         "provider": "com.xxx.test"
6       }
7     }
8   }
```

2. JS 调用原生方法

在 UniApp 页面的 `js` 文件中，通过 `uni.requireNativePlugin` 获取插件实例，调用方法：

Code block

```
1  // 获取插件实例
2  const testPlugin = uni.requireNativePlugin('com.xxx.test');
3
4  // 调用无回调方法
5  testPlugin.sayHello({
6    name: "UniApp 调用者"
7  });
8
9  // 调用带回调方法
10 testPlugin.getDeviceInfo({}, (err, res) => {
11   if (err) {
12     console.error("获取设备信息失败：", err);
13     return;
14   }
15   console.log("设备信息：", res); // {model: "iPhone", systemVersion: "17.0"}
16 });
```

五、调试插件（关键步骤）

1. 模拟器调试

1. 打开 HBuilderX → 选中 UniApp 项目 → 菜单栏「运行」→ 「运行到模拟器」→ 「iOS 模拟器」；
2. 模拟器启动后，执行调用插件的代码，Xcode 中可通过「Console」查看原生日志；
3. 若需修改原生代码：
 - 打开 Xcode 工程，修改代码后「Command + B」编译；
 - HBuilderX 中重新运行到模拟器（无需重新打包插件）。

2. 真机调试

1. 准备：iOS 真机连接 Mac → Xcode 中选择真机设备 → 登录开发者账号（需付费 Apple Developer 账号）；
2. 步骤：

◦ Xcode 中编译插件工程，生成 `.framework` 文件；

◦ HBuilderX 中「运行」→「运行到手机或模拟器」→ 选择真机；

◦ 真机上操作 UniApp 页面，触发插件调用，Xcode 可断点调试原生代码。

3. 常见调试问题

问题现象	原因分析	解决方案
插件找不到 (<code>requireNativePlugin</code> 报错)	<code>plugin.json</code> 标识/类名错误	核对 <code>plugin.json</code> 中 <code>id</code> 、 <code>class</code> 与原生类名一致
方法调用无响应	原生方法未加 <code>@objc</code> (Swift)	Swift 方法必须加 <code>@objc</code> 修饰
模拟器崩溃	依赖的框架不支持模拟器	选择支持模拟器的 SDK 或改用真机调试

六、打包与发布插件

1. 本地打包插件（供自己项目使用）

1. HBuilderX 中打开插件工程 → 菜单栏「发行」→「原生插件打包」→「iOS 插件」；
2. 选择打包模式（「调试包」/「正式包」），填写开发者账号（正式包需）；
3. 打包完成后生成 `.aar` (Android) / `.framework` (iOS)，复制到 UniApp 项目 `nativeplugins` 即可。

2. 发布到 DCloud 插件市场（供他人使用）

1. 插件工程根目录的 `package.json` 补充完整信息：

Code block

```
1  {
2    "name": "uni-test-plugin",
3    "version": "1.0.0",
4    "description": "iOS 测试插件",
5    "keywords": ["iOS", "插件"],
6    "author": "xxx",
7    "dcLoudext": {
```

```
8      "category": "功能扩展",
9      "platforms": ["ios"],
10     "id": "com.xxx.test"
11   }
12 }
```

2. HBuilderX 中「发行」→「发布到插件市场」→ 登录 DCloud 账号 → 填写插件信息并提交审核；
3. 审核通过后，其他开发者可通过插件市场导入使用。

七、进阶技巧与注意事项

1. 生命周期适配

若插件需感知 UniApp 页面生命周期（如 `onShow` / `onHide`），可在原生类中实现对应方法：

Code block

```
1  - (void)onAppShow {
2      NSLog(@"UniApp 前台显示");
3  }
4
5  - (void)onAppHide {
6      NSLog(@"UniApp 后台隐藏");
7  }
```

2. 内存管理

- 避免在插件中持有 `UIViewController` 强引用，防止内存泄漏；
- 回调 `callback` 后及时置空，避免多次回调。

3. 权限申请

若插件涉及隐私权限（如相机、定位、相册），需在 UniApp 项目的 `info.plist`（iOS）中添加权限描述：

Code block

```
1  <!-- 示例：定位权限 -->
2  <key>NSLocationWhenInUseUsageDescription</key>
3  <string>需要获取您的位置信息以提供服务</string>
```

4. 版本兼容

- 针对不同 iOS 版本做适配（如 iOS 14+ 相册权限、iOS 16+ 通知权限）；

- 插件需兼容 UniApp 不同基础库版本，避免桥接层 API 变更导致失效。

总结

iOS 原生插件开发的核心是「遵循 UniApp 桥接规范 + 原生 iOS 开发能力」：先通过 `plugin.json` 声明插件入口，再编写继承 `UniModule` 的原生类实现业务逻辑，最后在 UniApp 端调用并调试。重点需注意**通信参数格式、回调逻辑、权限适配**，以及 Swift 与 Objective-C 的兼容处理。掌握该流程后，可实现任意 iOS 原生能力在 UniApp 中的复用。

（注：文档部分内容可能由 AI 生成）